National University
Of Computer and Emerging Sciences

Foundation of Advancement
Of Science and Technology

Lab Number:	01
Course Code:	CL2005
Course Title:	Database Systems - Lab
Semester:	Spring 2026
Agenda:	Introduction to Databases, DBMS, SQL, and SQLite
Instructor:	Muhammad Mehdi
Email:	muhammad.mehdi@nu.edu.pk
Office:	Rafaqat Lab

Table of Contents

1 What is a Database?

A database is an organized collection of data that is stored in a structured form so that it can be efficiently retrieved, searched, updated and managed.

It can be:

- a binary file managed by a DBMS
- a set of files forming a database instance
- a single file database such as SQLite
- a distributed or in memory data store in non relational systems

A database focuses on three concepts:

1. Structure
2. Reliability
3. Efficient access

1.1 Flat-File Databases

A flat file database is a simple **plain text** or **CSV** file that stores data without relationships or constraints.

Example:

1, Ali, CS, 3.2
2, Maham, IT, 3.6

Limitations:

- no guaranteed data validation
- no concurrency support
- no indexing
- no ACID guarantees
- difficult to scale

Relational databases were created to solve these limitations.

1.2 Databases Vs Flat-Files

Text files and spreadsheets are simple but very limited and inefficient.

Databases offer:

1. Consistency through constraints
2. Speed through indexing
3. Multi user access
4. Data integrity (no partial writes, no corruption)
5. Security
6. Query power through SQL
7. Relationships among entities
8. Transactions to group operations safely

Flat files are good for small isolated tasks. Databases are required for reliable applications, websites, enterprise systems and data analysis.

2 What is a DBMS?

A DBMS is a software system that creates, stores, manages and retrieves data in a database. It provides rules, file formats, indexing methods, query processing, transactions and concurrency control.

There are two major categories.

1. Relational DBMS (RDBMS)
2. Non Relational DBMS (NoSQL)

2.1 Relational DBMS (RDBMS)

An RDBMS stores data in **tables** with **rows** and **columns**. It uses **SQL** as the standard language.

Examples:

- MySQL
- PostgreSQL
- Oracle
- Microsoft SQL Server
- SQLite (lightweight but still relational)

RDBMS properties:

- Structured data
- Fixed schema
- ACID (Atomicity, Consistency, Isolation, Durability) transactions
- Relationships using primary and foreign keys

2.2 Non-Relational DBMS (NoSQL)

A NoSQL DBMS stores data in non relational formats. There is no single standard structure or language.

Popular types:

- Document databases: MongoDB
- Key value stores: Redis
- Column stores: Cassandra
- Graph databases: Neo4j

NoSQL is used when:

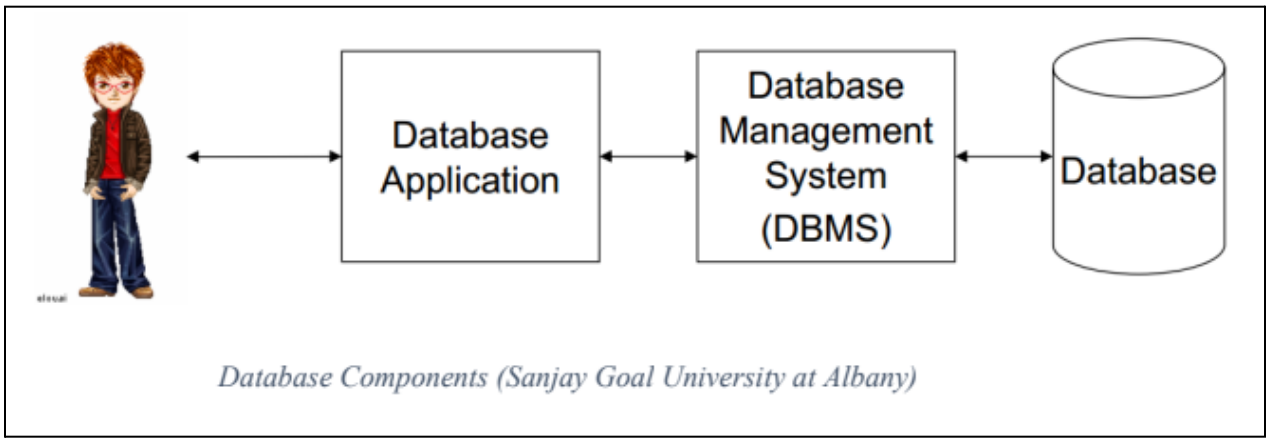
- data is semi structured or unstructured
- massive horizontal scaling is required
- schema flexibility is needed

3 What is a Database System?

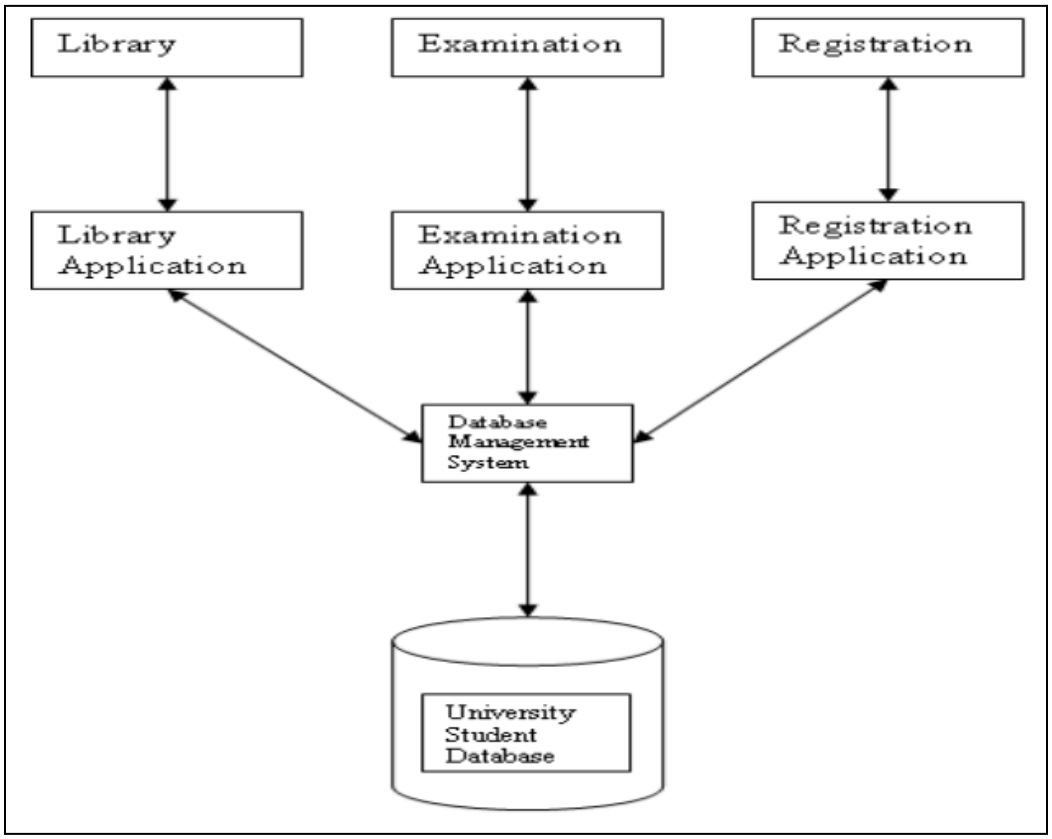
A database system is a **complete environment** consisting of a database, DBMS, and application programs used to store, retrieve, and manage data efficiently.

It consists of:

- **Database:** the actual stored data
- **DBMS:** the software that manages the database
- **Applications:** programs that interact with the database through the DBMS
- **Users:** people or systems that use the data



Database Components (Sanjay Goal University at Albany)



4 What is SQL?

SQL stands for **Structured Query Language**. It is a domain-specific **declarative** programming language. It serves as a standard language to interact with relational databases.

SQL is used to:

- create databases and tables
- insert data
- update or delete data
- query and filter rows
- control permissions
- define constraints

Every RDBMS provides its own SQL engine but the core language concepts remain the same.

3.1 Basic SQL Syntax Rules

- Statements end with a semicolon
- Keywords are case insensitive
- Strings are in single quotes
- Identifiers with spaces need double quotes. Identifiers without spaces need no quotes.

3.2 What is a Schema

A schema is the structure of the database. The schema is essential for designing the database before inserting any data.

It defines:

- tables
- columns
- data types
- constraints
- relationships

5 What is SQLite?

SQLite is a lightweight relational database engine. It is one of the most widely used and simplest RDBMS.

Characteristics:

- Serverless
- Zero configuration
- All data is stored inside one file such as `database.db`

- Uses standard SQL
- ACID compliant
- Extremely fast

SQLite is excellent for:

- learning SQL
- mobile apps
- small desktop apps
- local storage
- prototyping

SQLite is different from heavy DBMS (MySQL, Oracle etc) because:

SQLite	Traditional DBMS
Single file database	Multi file database server
No server process required	Requires a running server
Very easy to install	More complex setup
Ideal for developing small scale apps	Ideal for large scalable systems

6 Setting Up SQLite

6.1 Windows Setup

- Go to https://www.sqlite.org/download.html
- Download sqlite-tools zip file
- Extract to a folder such as `C:\sqlite`
- Add that folder to PATH (optional)
- Open Command Prompt and run:

```
sqlite3 --version
```

6.2 Linux / Ubuntu Setup

1. Open Terminal and enter:

```
sudo apt update
sudo apt install sqlite3
```

2. Now run:

```
sqlite3 --version
```

7 DB Browser for SQLite

DB Browser for SQLite is a graphical tool that allows you to interact with SQLite databases through a graphical user interface.

Following features are available in DB Browser for SQLite:

- create SQLite databases
- create and edit tables
- insert or browse data
- run SQL queries
- view schema visually

Download from: https://sqlitebrowser.org

8 Getting Started with SQLite

8.1 Creating a Database

- Open terminal or command prompt and type:

```
sqlite3 example.db
```

- This creates a new database file or opens it.
- Your prompt changes to the SQLite prompt in which you can now execute SQL statements:

```
sqlite>
```

8.2 Useful Dot Commands

These commands start with a dot (`.`) and are specific to SQLite CLI. They are **not part of standard SQL** and only work inside the SQLite shell.

Command	Purpose
`.help`	Display a list of all available dot commands with brief descriptions.
`.open filename`	Open the specified database file (creates it if it does not exist).
`.databases`	List all currently attached databases.
`.tables`	Show all tables in the current database.
`.schema tablename`	Display the schema (table structure) of the specified table.
`.mode table`	Display query results in a formatted, boxed table view.
`.read filename`	Execute all SQL commands from the specified file (e.g., `.read commands.sql`).
`.system command`	Execute a system/OS command from within the SQLite shell (e.g., `.system cls`).
`.exit`	Exit the SQLite shell.

8.3 SQLite Data Types

SQLite uses a flexible typing system based on storage classes. SQLite uses **dynamic typing**, so the type is more of a "suggestion" than a strict rule.

Storage Class	Description / Use
`INTEGER`	Whole numbers (positive, negative, zero)

`REAL`	Floating-point numbers (decimal numbers)
`NUMERIC`	Numbers that can store integers or decimals; SQLite will try to store values as `INTEGER` or `REAL` if possible
`TEXT`	Text or string data (UTF-8, UTF-16)
`BLOB`	**B**inary **L**arge **O**bject is used to store binary data (images, files, or any raw bytes); stored exactly as inserted

Some other names like VARCHAR, FLOAT, BOOLEAN are accepted but internally mapped to these categories.

8.4 Basic Constraints

Constraints are rules applied to table columns that **restrict the type of data that can be stored**, ensuring data integrity, accuracy, and consistency.

They prevent invalid or duplicate data and help maintain reliable databases.

Constraint	**Description / Use**
`PRIMARY KEY`	Uniquely identifies each row; cannot be NULL; for `INTEGER PRIMARY KEY`, it can alias `rowid`
`NOT NULL`	Prevents NULL values in a column
`UNIQUE`	Ensures all values in a column (or combination of columns) are distinct
`DEFAULT`	Sets a default value when no value is provided during `INSERT`
`CHECK`	Ensures that column values satisfy a condition (e.g., `CHECK(cgpa <= 4.0)`)

8.5 Example: Creating a Table with Data Types and Constraints

The following SQL statement demonstrates how to create a table named `students`, specifying appropriate data types and constraints for each column.

```
CREATE TABLE students (
    id      INTEGER   PRIMARY KEY,
    name    TEXT      NOT NULL,
    email   TEXT      UNIQUE,
    dob     NUMERIC,
    cgpa    REAL      DEFAULT 0.0 CHECK (cgpa <= 4.0)
);
```